

AI-ASSISTED DEVELOPMENT BRIEF

I don't type the code.
I design it, I steer it,
I sign it off.

Keepr is a memory-messaging app in production: text, photo, voice, GIF — everything is kept. No monetisation, no harvesting, no feed. Nine tabs, one conversation. Designed, steered and maintained alone, with AI-assisted development, fully remote — without being a trained software engineer.

WHAT THIS IS WORTH ON ASSIGNMENT

Every reflex here is a BA reflex: scope before execution, traceability, producer / reviewer separation, quality of the steering data — with evidence attached.

PWA React

in production
daily use

9 tabs

a single
conversation

166 tests

Firestore rules
rerun on every deploy

1 designer

fully remote
no line-by-line review

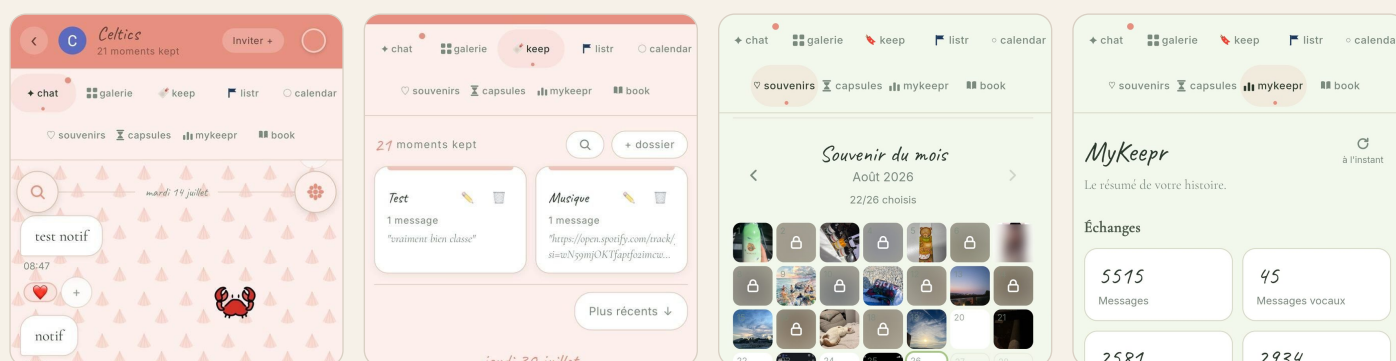
Mathieu

BUSINESS ANALYST · PRODUCT OWNER

2026

PWA React · production · real users

AI-assisted development



01 Everything lives in the thread

Keepr is not a feed that scrolls. It is a shared memory, with nine doors.

THE SIGNATURE GESTURE

Long-press a message, slide it to Keep: it becomes a polaroid, kept for good. Editing the message never rewrites the keep. The gesture became a habit — the best adoption signal I have.

Chat

One-to-one or group threads, voice notes, reactions, threaded replies, live typing.

Keep

The core: messages kept as polaroids, filed in folders. A keep is immutable.

Calendar

Key dates, birthdays and dated memories on a single view.

Capsules

A message sealed today, opened on a chosen date, with a notification on the day.

Book

A memory book: photos laid out as pages, freehand notes, printable PDF export.

Gallery

Media sorted by type; search by place, month or face (beta); a map.

Listr

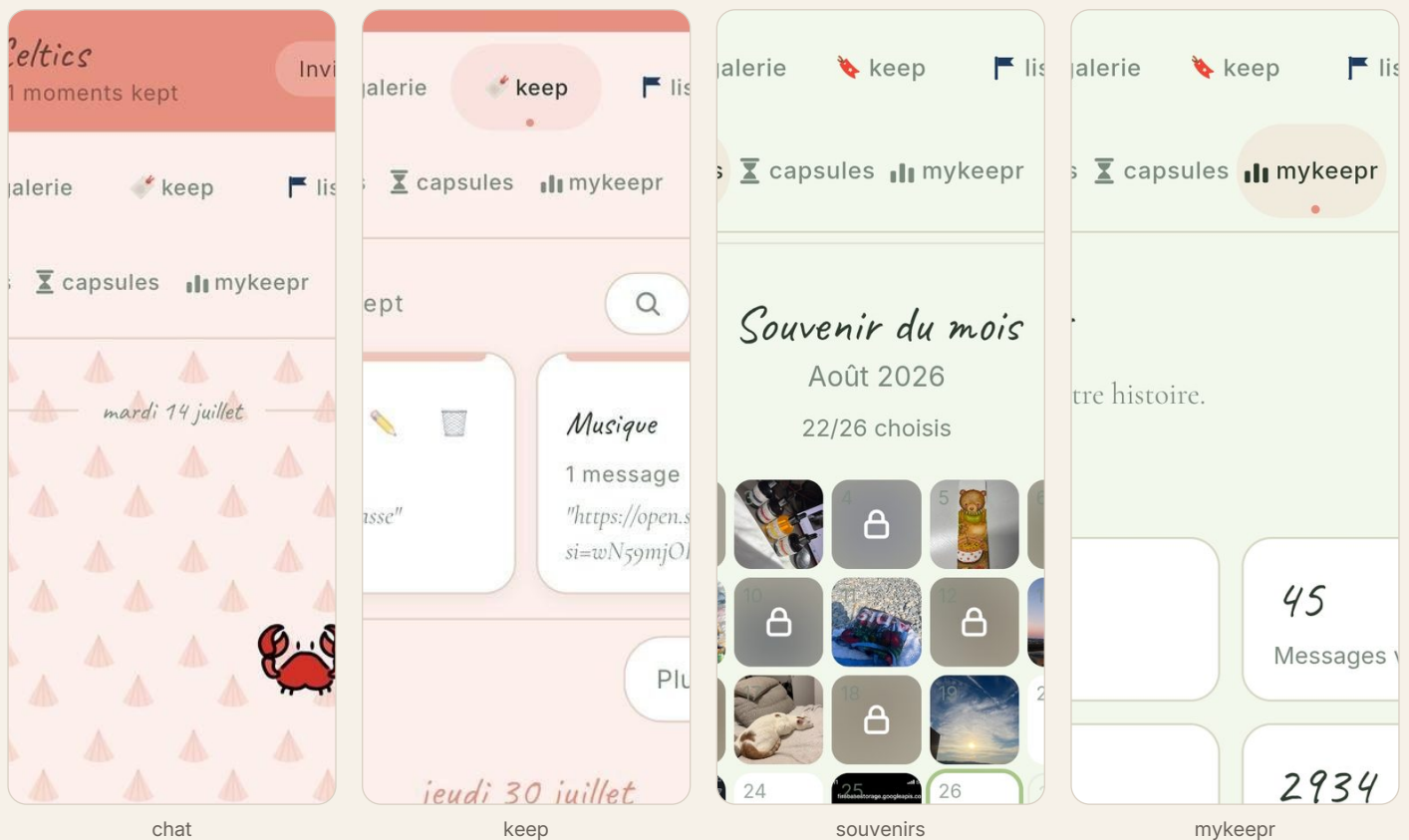
Outing ideas in three lanes: idea, upcoming, past. From wish to memory.

Souvenirs

Automatic daily montage, an "Our story" feed, a monthly memory to compose.

MyKeepr

A counted summary of the thread: volumes by media type, age of the conversation.



THE RULE

Nine entries, one conversation, no data leaving the circle. No roadmap feature may require a new paid infrastructure brick. Billing cut-off armed and tested, isolated service accounts, zero data warehouse, versioned rules in the repo. The ceiling is set at scoping, before the feature.

02 Steer, don't type

The skill is not asking an AI for code. It is knowing what to ask, in what order, and recognising when the answer is wrong. No one signs off their own work.



WHAT I DO

- Design and settle architecture before any execution.
- Write self-contained briefs: context, constraint, done-when.
- Have the report challenged by an instance that did not produce the code.
- Sign off on real use and material evidence.

WHAT I DON'T

- Write or review code line by line: that is not my review.
- Take an agent report at its word.
- Confuse "the build is green" with "it works".
- Let a session run until the context decays.

WORK CYCLE — THE COST SITS IN THE PLAN, NOT IN THE EXECUTION

1 Design · 2 Recon of what exists · 3 Self-contained brief · 4 Test and non-regression · 5 Adjust or merge

Three pieces of evidence, on Keepr

Not stories. Runs, dates, gaps left visible.

CRITICAL · SECURITY

Invites: an authenticated outsider could join a conversation.

Abuse tests on the authorisation model found it. The lock is closed and replayed on every deploy. Adjacent points still open stay deliberately red, prioritised, with their trigger. The run is green in the CI sense — not in the "everything is fixed" sense.

Run firebase-deploy #37 · 23 Aug 2026 · commit 213919c · project keepr-529cc · 11 rules files · 166 tests · 162 passing · 4 expected failures.

CRITICAL · COST

A cloud bill that can explode overnight.

Pay-as-you-go: an infinite loop, a burst of writes, and the invoice runs with no ceiling. For a project with no revenue, that is the first risk. Automatic cut-off: alert, queue, then detach the billing account. Staged thresholds, tested in simulation, then armed.

Job inspect-cost-daily · run firebase-inspect #47 · 26 Aug 2026 · cap EUR 5 · threshold not crossed · last receipt 19:38:57 UTC.

STRUCTURAL · AGENT TRUST

When the agent claims to have done what it did not.

Execution reports that are fluent, confident, and sometimes false: announced commits and files that were not there. No report is accepted without evidence — repo state, history, listing. That is the differentiating skill of working with agents.

Confrontation, 26 Aug 2026: "commits produced: none; files touched: none." History confirms it. The control holds by being systematic.

03 What this work demonstrates

Scope before you execute. Five reflexes — the ones that change something for a client.

ON KEEPR — THE EVIDENCE

ON ASSIGNMENT — WHAT IT YIELDS

01	AI ACT HUMAN OVERSIGHT	The person who produces is not the person who checks. The agent report is challenged by a distinct instance. No conclusion is kept without material evidence. A security fix stayed in staging until the test was replayed.	Put a producer / reviewer split on an AI-assisted process, with auditable control points — not a “validated by a human” box that no one can reconstruct.
02	BA CRAFT UPFRONT SCOPING	Every piece of work starts with a settled design — perimeter, expected behaviour, edge cases, done-when — and a recon of what exists before any brief.	The job itself: lift ambiguity upstream instead of finding it in UAT. A specification that holds kills build errors before they are coded.
03	AI ACT LOGGING	Every admin access attempt, allowed or denied, is logged. Administration is restricted to a single account.	Specify usable logging — know what you must be able to reconstruct, and ask engineering for it before go-live, not after the incident.
04	GDPR MINIMISATION	A 30-day purge treated as non-optional. Telemetry designed with no message content. Zero data warehouse.	Write and hold retention and minimisation requirements against a business that wants to keep everything “just in case”.
05	AI ACT DATA LIMITS	Median shown next to the mean where the mean alone was lying. A biased metric found in my own telemetry, documented, then pulled from decisions.	Challenge a report before it becomes an argument — document what a system cannot do, rather than let a flattering indicator decide in place of the business.

LIMITS I OWN

I am not a trained software engineer and I do not claim to be. I do not write the code and I do not review it line by line: I design, I have it produced, I have the report contradicted, I decide on evidence. The security tests described here are mine, on my own product — this is not an external audit. Keepr carries identified, numbered, prioritised debt, each item with its trigger. An isolated development environment does not exist yet: that is written and prioritised, not ignored.

Mathieu

Business Analyst · Product Owner · August 2026

Provenance — no figure is estimated. Screenshots from Keepr in production; personal data masked.